Interviews and Stripe's Operating Principles

Stripe's operating principles guide how we interact with each other and our users, translating our values and beliefs into concrete actions. The principles articulated here detail the everyday behaviors we focus on, our core traits, and what we expect of our leaders—and all are rooted in the core expectation that Stripes operate with high integrity, maturity, and respect for others at all times.

How we work	Who we are	And leaders…
Users first. We have a weighty obligation to the businesses built on Stripe and the everyday people they serve. Because we're so critical to our users' success, we must keep their needs front and center in all we do.	**Curious.** We lead with a genuine interest in people, ideas, and the unknown. We work hard to understand other points of view, and prefer investigating to being right.	**Obsess over talent.** The quality of the Stripes you attract and retain defines your team, so developing an unusually keen talent radar can be your biggest competitive advantage. Be the person who reads and investigates every candidate packet.
Move with urgency and focus. A bias for action speeds our learning and delights our users. Focus on what matters most, make fast initial progress, and iterate toward the best outcome.	**Resilient.** Startups are tumultuous places and the audacity of our goals will mean occasional failure. We view setbacks as opportunities to sharpen our skills.	**Elevate ambitions.** Know where your team is headed next year/in 4 years/in 10 years, and develop a compelling vision for that future. Guide your team to redefine what's possible by expanding what's reasonable.
Be meticulous in your craft. Doing things well is in Stripe's DNA. We value craftsmanship for its own sake, and are fervently committed to producing surprisingly great work.	**Humble.** Stripes are humble, not arrogant or complacent, and create an inclusive environment for all. We aren't wedded to how we currently do things—lots about our current practices will turn out to be wrong.	**Set the pace and energy.** Slow, incremental progress is not an engaging org strategy—Stripe is a marathon of sprints.
Seek feedback. We value intellectual honesty and look for other Stripes with the expertise to refine our ideas, challenge us, and deepen our understanding across the business.	**Macro-optimistic.** Stripes reject cynicism, knowing that all problems can be solved with the right understanding, and progress is only inevitable through focused effort. We are believers in the long term.	**Make decisions; be accountable.** It's not always clear who should make a decision. Effective leaders embrace decision-making in cases of murky ownership, either making the call themselves or collaborating across the necessary teams to drive an outcome. They clearly communicate decisions and hold themselves and their teams accountable for results.
Deliver outstanding results. Stripes are high achievers with a drive to succeed. We take end-to-end accountability for seeing our work through and delivering on our commitments.	**Exothermic.** Stripes generate an energy and warmth that is infectious across teams and throughout the company. We are genuinely excited about our work, and about creating an exceptionally welcoming environment for all Stripes.	**Lead with clarity and context.** Translate chaos into a clear, compelling plan. Be deeply informed about what's happening across Stripe, and create your team's plans in reference to the broader work.
		Solve problems. Be a persistent force for progress. Our leaders must work with their teams and across Stripe to quickly and effectively solve problems—especially when they're hard.

Behavioral Interviewing with STAR

Wherever you see 'Operating Principle Interview' referenced as part of the interview process, you should use the following framework and tips to guide your response.

Situation/Task: explain the situation or task so the interviewer understands the context of your example

Action: provide details on what you (and/or a teammate, partner, or stakeholder) did to address the situation or carry out the task

Result: describe what your actions achieved and the ways in which you were effective, successful, unsuccessful, impactful, transformative, notable, etc.

Tips:

- Be detailed, yet concise. Interviewers will ask follow-up questions when they need more details, but it's best to stick to the STAR format in order to cover the who, what, where, why, and how behind your response
- Never feel obligated to share details, metrics, people, or initiatives that are confidential or proprietary. If you need to redact sensitive details from your response, just explain this to the interviewer
- Use "I" instead of "we" when speaking in first person about something that you individually owned, a decision that you individually made, or other instances where it's important to clarify this to the interviewer. "We" can be used to represent the work of the broader team or a group of colleagues.
- Specific examples help interviewers get a clear picture of who you are and how you work. Avoid generalizations ("this happens a lot, so I usually…") or hypotheticals ("I would…")
- Try not to repeat examples; the more variety you share, the better your interviewers will understand the complexity and scope of your work
- Ask clarifying questions. Interviewers are happy to make sure you understand what is being asked or re-frame a question that may have been unclear

General Tips

Many of the interviews included during your process are designed to help us understand how you might approach common scenarios encountered at Stripe. Most of the exercises you'll do in these interviews will be directly related to Stripe. However, you're not being tested for your knowledge of Stripe and you'll always have the opportunity to ask us questions about anything you don't understand.

HOW TO PREPARE

- Refer to our Stripe Video Interviewing What to Expect document.
- Take a look at our blog and Twitter.
- Do some background reading about our products and documentation. This will provide some context for the questions we'll ask in the interviews.
- Be well-rested and ready to think on your feet, as some of the interviews will require you to problem-solve in real time.
- Familiarize yourself with our Operating Principals.

WHAT WE EVALUATE

We want you to have the opportunity to present your best self. The detailed descriptions of each interview contain specific things we evaluate for each specific session. Here is what we evaluate overall:

- **Problem solving**
  - How effective are you at understanding the problem and devising a solution for it?
- **Design**
  - How do you design and lay out your code? Is it well organized with easy to understand interfaces?
- **Correctness**
  - Do you think about the correctness of your code? Do you use explicit reasoning and/or tests to check correctness? Do you consider edge and error cases?
- **Debugging**
  - Do you spot the bugs in your code? When you find one, can you fix it?
- **Programming language familiarity**
  - Are you comfortable with the language you're writing? Is your code idiomatic for the language?
- **Tools familiarity**
  - Are you set up to write and run code? (You don't need an advanced editor or setup but you should be comfortable and productive in your chosen environment.)
- **Communication**
  - Do you clearly explain your thoughts? How well do you respond to feedback or suggestions from your interviewer? Do you seek help or guidance when stuck?

Technical Interviews

In these interviews, you'll consider technical problems and then write working programs that solve them. In some cases, you'll also have a design discussion with your interviewer before diving into coding. Our technical interviews simulate the engineering work you'd do day-to-day at Stripe.

We want you to use whichever programming language, environment, and editor you're most comfortable with. Please choose a language that you are fluent with, as some questions ask you to work with a mature codebase, and choosing a language you can't write easily will make it hard to correctly assess your abilities. If you are using an unfamiliar IDE or runtime environment, take some time prior to the interview to acquaint yourself with how to run and debug code.

We encourage you to use online resources (docs, Stack Overflow, etc.) as needed during the interview. **Please note that the use of any AI tools (e.g., Copilot, Gemini, TabNine, ChatGPT, LLMs, or any other AI assistants) in solving interview problems is strictly prohibited. This includes the use of AI results in search tools, such as Gemini for Google Search.**

Consider your interviewer a collaborator on the problem. We suggest "working out loud" with your interviewer and asking any questions that you have as they come up, for example about constraints on the system.

HOW TO PREPARE

- You will need to use a computer (Mac, Windows, or Linux) that is set up for software development. You should not use your current employer's computer.
- We ask questions that require writing, running, and debugging code in your language of choice on your computer. You should have:
    - A text editor or IDE you are comfortable with
    - The ability to run a program and print its output
    - We recommend completing the steps in "Environment Setup" below.
- While your previous interview may have used an online coding platform such as HackerRank or CoderPad, the questions we ask during the virtual onsite will require compiling, editing, and running an open-source project on your computer. We encourage using debugging tools, built-in data structures, and any open source libraries/techniques at your disposal. Again, we strongly recommend cloning and executing the repositories in "Environment Setup".
- One of the interview questions will be structured around HTTP requests. You should have a working familiarity of an HTTP client of your choice in the language you are interviewing in. You should expect to be making HTTP requests to a public API and making use of the responses. A high-level understanding of this article is a good reference.
- Come with questions for your interviewers. These can be technical or not. Think about what's important to you, and ask us how well Stripe does on those things. Don't be afraid to ask hard questions. Julia Evans put together a great blog post on questions for interviewers.

ENVIRONMENT SETUP

Be prepared to clone a private repository from GitHub. This will require you to have a GitHub account and that you have your authentication set up for cloning a private repository. During your interviews, you will be able to "Download ZIP" (which doesn't require SSH or password-based authorization), in case you haven't set-up your SSH keys in advance.

We ask that you set up a local dev environment on your machine, as you will be screen sharing at points during the onsite.

If you're interviewing in one of the languages below, we have some specific setup instructions you should use to prepare in advance. Some of the languages below contain tests, in addition to build instructions, to verify the environment setup is working. Please ensure you are able to run these tests prior to your interview.

If you are using a Mac and have Homebrew installed, you will be in good shape.

- **Ruby**
  - We've created a Ruby project that will ensure your computer is configured correctly to write Ruby for your interview. Before the interview, please clone the repo and follow the setup instructions. The setup will require you to install Ruby on your laptop (a version later than 2.3.0), as well as Bundler to manage the dependencies of your code.
- **Java**
  - Please run this Maven project to confirm your computer is configured the way our questions expect. The repository has suggestions under the "Getting Ready" section.
- **Python**
  - We've created a Python project to help you determine if your computer is configured correctly to write Python 3 for the purposes of our interview.
- **Scala**
  - We've created an SBT project to help you determine if your computer is configured correctly to write Scala. Before your interview, please visit our repository.
- **C#**
  - We've created a C# ProjectSolution to help you determine if your computer is configured correctly to write C#, at least at a level that some of our questions expect. Before your interview, please visit the repository and follow the instructions under the "Getting Ready" section.
- **C++**
  - We've created a C++ project to help you determine if your computer has the required software and is configured correctly to write C++, at least at a level that some of our questions expect. Before your interview, please visit the repository and follow the instructions under the "Getting Ready" section.
- **Javascript**
  - We've created a JavaScript project to help you determine if your computer has the required software and is configured correctly to write JavaScript, at least at a level that some of our questions expect. Before your interview, please visit the repository and follow all the instructions in the README.md file.

- **Go**
    - Make sure you have a supported version of Go installed on your laptop. Go supports the 2 most recently released major versions, which can always be found at: https://go.dev/dl/
    - Any Go-capable editor or IDE is fine for interview purposes. Whatever you choose, ensure that you have a setup that lets you easily accomplish the following tasks:
        - Open an existing Go module directory and compile its contents
        - Run “go install” to compile a binary and its dependencies (example: go install golang.org/x/tools/cmd/goimports@latest)
        - Run a basic “Hello World!” program (example)
    - For the Bug Squash interview, you’ll be debugging an existing Go package. If you like to run an interactive debugger like Delve or IntelliJ/GoLand, ensure that your debugger of choice is working prior to the interview. “Print style” debugging (with fmt.Printf statements) is fine too if that’s your preference!
- **Other Languages**

Make sure you can compile and run a simple Hello, World! application, using the relevant dependency management system for your language.

Interview Process Overview (Pre-Onsite Stage)

Programming Exercise Interview [Functional]

What to expect & how to prepare

In this 45 minute interview, you will meet with a Stripe Software Engineer who will look to understand your ability to write correct, idiomatic code in a reasonable amount of time. Be prepared to discuss a realistic and in depth programming exercise.

This will be a generic coding exercise

In these interviews, you'll consider technical problems and then write working programs that solve them. In some cases you'll also have a design discussion with your interviewer before diving into coding.

You should use whatever programming language, environment, and editor you're most comfortable with. Please do not just choose a language that is good for algorithm demonstration - you will actually be working in a real codebase. We encourage you to use online resources (docs, Stack Overflow, etc.) as needed during the interview. Please note that the use of any AI tools [e.g. Copilot, etc.] in solving interview problems is strictly prohibited.

Consider your interviewer a collaborator on the problem. We suggest "working out loud" with your interviewer and asking any questions that you have as they come up, for example about constraints on the system.

What we're evaluating

In this interview we will be looking to get a sense for the following:

- Interaction and Collaboration
    - Clearly explained thought process, responding constructively to feedback or questions, and asking questions of your own.
- Thought Process and Abstractions
    - Come up with great abstractions, interfaces, and pick appropriate data structures. Find and understand the information you are searching for.
- Writing Code
    - Write code that is very readable. Show your skill in a particular language/area, and be knowledgeable about tools and your development environment.
- Correctness, Testing, and Debugging

Think about correctness , and debug the code with purpose. Write repeatable test cases that exercise edge cases or catch regressions.

Design 1 Interview [Functional]

What to expect & how to prepare

In this 60 minute interview, you will meet with a Stripe Software Engineer who will look to understand how you understand design systems and organize and communicate your technical thoughts aligned to your area of expertise (General Engineering, ML, Frontend, Mobile). Consider your interviewer to be a collaborator on the problem and plan to work with them throughout the session. We suggest “working out loud” with your interviewer and asking any questions as they come up, such as constraints on the system. This interview doesn’t involve coding, nor will it be an esoteric technical problem that requires pre-existing knowledge of Stripe.

We care that you: understand the problem, make decisions and discuss associated trade-offs, adapt your design, and communicate well. We do not care about specific technologies nor are we looking for one specific solution (there are many!). Be sure to justify and explain your decisions.

Since you will be completing this interview over VC instead of in-person, you can share your design with the interviewer using one of the following tools:

- Whimsical collaboration tool (preferred)
- Zoom’s whiteboarding tool
- Any preferred drawing tool that is installed on your device, or even a physical whiteboard if you have one handy at home

Feel free to use this resource to familiarize yourself with - and get set up using - the Zoom or Whimsical collaboration tool.

What we’re evaluating

In this interview we will be looking to get a sense for the following:

- Design Approach
    - Describe the components of design and explain design decisions and tradeoffs. Bias towards simplicity, and make sure to justify decisions. Be decisive and clear about what you’re choosing.
- Quality of design
    - The design meets the functional requirements and addresses some non-functional requirements.
- Interaction and collaboration
    - Explain the thought process during the interview and respond to feedback from the interviewer.
    - Be decisive and clear, but willing to iterate design with additional information for feedback.